

BitBatSPIR: Efficient Batch Symmetric Private Information Retrieval from PSI

Shuaishuai Li[✉], Liqiang Peng[✉], Weiran Liu[✉], Cong Zhang[✉], Zhen Gu[✉], and Dongdai Lin[✉]

Abstract—Private Information Retrieval (PIR) allows a client to retrieve an entry from a database held by a server without leaking which entry is being requested. Symmetric PIR (SPIR) is a stronger variant of PIR with database privacy so that the client knows nothing about the database other than the retrieved entry.

This work studies SPIR in the batch setting (BatchSPIR), where the client wants to retrieve multiple entries. In particular, we focus on the case of bit entries, which has important real-world applications. We set up the connection between bit-entry information retrieval and set operation, and propose a black-box construction of BatchSPIR from Private Set Intersection (PSI). By applying an efficient PSI protocol with asymmetric set sizes, we obtain our BatchSPIR protocol named **BitBatSPIR**. We also introduce several optimizations for the underlying PSI. These optimizations improve the efficiency of our concrete BatchSPIR construction as well as the PSI protocol.

We implement **BitBatSPIR** and compare the performance with the state-of-the-art PIR protocol in the batch setting. Our experimental results show that **BitBatSPIR** not only achieves a stronger security guarantee (symmetric privacy) but also has a better performance for large databases, especially in the Wide Area Network (WAN) setting.

Index Terms—Private information retrieval, black-box construction, private set intersection, BitBatSPIR.

I. INTRODUCTION

PPRIVATE Information Retrieval (PIR) is a specific secure two-party computation protocol run between a server and a client. PIR aims to ensure that the client can retrieve an entry from a database while preventing the server from obtaining any information about the retrieved entry. PIR has a wide range of applications due to its security assumption, such as private

communication [1]–[3], certificate transparency [4], private media consumption [5], and password checkup [6].

Since the concept and the first construction was introduced in [7], [8], a long line of work focused on improving the efficiency of PIR. In this paper, we focus on single-server PIR, which assumes that the client can only access a single server. Earlier number-theoretic PIR protocols [9]–[12] achieve poly-logarithmic communication, but they incur high computational cost, which makes them less efficient in practice than simply having the server send the entire database to the client [13]. This situation was changed by the work of [14], which designed the first practical PIR protocol using lattice-based homomorphic encryption. Following this work, many lattice-based PIR families [3], [6], [15]–[19] achieving various efficiency features have been presented.

Symmetric PIR (SPIR), introduced by [20], is a stronger variant of PIR. SPIR additionally requires database privacy. That is, the client cannot know anything about the database except its retrieved entry. Although many works have been devoted to constructing efficient PIR, little is about SPIR. The work of [6] recommended using Oblivious Pseudo-Random Function (OPRF) to achieve database privacy, but they did not provide any implementation. More recently, the work of [21] designed and implemented a SPIR protocol with efficiency comparable to the state-of-the-art PIR protocols.

In some applications, the client may want to retrieve multiple entries at once from the database. A typical example is that a user retrieves multiple messages addressed to her from an anonymous messaging system [22]. This is referred to (S)PIR in the batch setting. Although one can simply invoke (S)PIR multiple times to realize such functionality, designing specific protocols would amortize the cost of per query. Several works [3], [22]–[24] have studied PIR in this setting. The work of Vectorized Batch PIR (VecBatchPIR) [24] names PIR in the batch setting as BatchPIR, and proposes the state-of-the-art protocol for database with small entries. To the best of our knowledge, there are currently no publicly available implementations of SPIR in the batch setting, which we name BatchSPIR, following the convention.

A. Motivation

As pointed out by [24], one annoying feature of lattice-based (S)PIR protocols is that they only have decent performance when the entry is in a proper size. More specifically, the server needs to treat the entries as plaintexts in lattice-based encryption schemes, and typically the plaintext space is relatively large. Therefore, entries with sizes that just fit the plaintext

This work is supported by the National Natural Science Foundation of China (Grants No. 62372449), the National Key R&D Program of China (No. 2024YFA1013000), the Major Programs of the National Social Science Foundation of China (Grant No. 22&ZD147), the National Natural Science Foundation of China (Grant No. 62402272).

• Shuaishuai Li is with Zhongguancun Laboratory, Beijing, China. (email: liss@zgclab.edu.cn)

• Liqiang Peng and Weiran Liu are with the Department of Data Technology and Products at Alibaba Group, China. (email: plq270998@alibaba-inc.com, weiran.lwr@alibaba-inc.com)

• Cong Zhang is with Institute for Advanced Study, BNRist, Tsinghua University, Beijing 100084, China and State Key Laboratory of Cryptography and Digital Economy Security, Tsinghua University, Beijing, 100084, China. (email: zhangcong@mail.tsinghua.edu.cn)

• Zhen Gu is with DAMO Academy of Alibaba Group, China (email: guzhen.gz@alibaba-inc.com)

• Dongdai Lin is with State Key Laboratory of Cyberspace Security Defense, Institute of Information Engineering, Chinese Academy of Sciences, China and School of Cyber Security, University of Chinese Academy of Sciences, China. (email: ddlin@jie.ac.cn)

• Cong Zhang is the corresponding author.

space would take full advantage of the plaintext room under the same ciphertext size, resulting in a low communication overhead. This limits the use of these schemes in applications with small entries, especially if the entries are just *bits*. Consider the following two applications:

- *Private Drug Request.* A patient (named Alice) wants to know whether the hospital has several certain drugs. For privacy reasons, Alice does not want the hospital to know which drug she is looking for. For business reasons, the hospital does not want to give Alice information about the drugs other than that Alice asked about.
- *E-commerce Advertising.* An e-commerce company has inferences that a group of consumers (identified by IP addresses) may like to buy a certain product with the brand. The online brand seller wants to know if it needs to show the advertisement to a list of candidate consumers. The online brand seller is not allowed to leak the candidate consumers to the e-commerce company, while the e-commerce company may only want to reveal desired inference results to the online brand seller.

When the retrieval condition (drugs and IP addresses in a certain area in the above cases) can be publicly ordered, the problem can be formalized as BatchSPIR with bit entries. This leaves the following question:

Can we construct more efficient BatchSPIR with even bit entries?

B. Related Primitives

We first recall several related primitives before going into the details of our work, including Private Set Intersection (PSI), SPIR by keywords (Keyword SPIR), and Labeled PSI.

PSI. PSI [25], [26] is a two-party protocol where the sender and the receiver, each holds a private set, want to compute the intersection of the two sets without leaking anything else. A common setting of PSI is that the two input sets are of the same (or similar) size, and many works have been designed for better performance in this setting [27]–[38].

Although these PSI protocols are very efficient, their communication is linear in the sizes of both sets. This results in a significant increase in communication cost in an asymmetric setting where one set is much larger than the other one, compared to the insecure solution that has communication cost proportional to the size of the smaller set. Motivated by this, several works [29], [39]–[43] considered constructing practical PSI protocols in the unbalanced setting, where the size of the sender’s set is much larger than that of the receiver’s set. In particular, the PSI protocols presented in [40], [42], [43] achieve sublinear communication cost in the size of the larger set.

Keyword SPIR. The work of [44] introduced Keyword PIR, where the database is a set of n key-value pairs rather than a tuple with n values, and the input of the client is a search key rather than an index in $[n]$. The goal of Keyword PIR is to enable the client to obtain the value corresponding its search key. Similarly, we can extend SPIR to Keyword SPIR. Note that when the key set is $[n]$ where n is the number of the

entries in the database, Keyword SPIR (PIR) degenerates into SPIR (PIR).

Labeled PSI. PSI can be further extended to the concept of Labeled PSI [40], where the sender generates a label for each element in its set, and the receiver will obtain the labels of the elements in the intersection. When we treat the receiver’s input set as a keyword set and the sender’s labels as the corresponding values of the sender’s input elements, then labeled PSI is equivalent to Keyword SPIR in the batch setting (Keyword BatchSPIR) [42].

The relation between Keyword BatchSPIR and Labeled PSI naturally raises the following question:

Is there a similar relation between BatchSPIR and PSI?

A straightforward thought is whether we can derive any reduction between BatchSPIR and PSI from the equivalence of Keyword BatchSPIR and Labeled PSI. However, the answer seems not trivial. If we replace Keyword BatchSPIR with BatchSPIR, then in the resulting “Labeled PSI” protocol, the set of the sender will be $[n]$ and the set of the receiver will be a subset of $[n]$. This does not capture a PSI protocol because PSI does not specify the inputs. Moreover, if we replace Labeled PSI with PSI (where each element serves as its own label), then in the resulting “Keyword BatchSPIR” protocol, each key serves as its own value. This does not capture a BatchSPIR protocol because BatchSPIR only limits the key set to $[n]$, without restricting the relationship between the key set and value set.

C. Our Contributions

In this work, we answer these two questions affirmatively. Our contribution can be summarized as follows.

Black-Box Construction of BatchSPIR from PSI. The starting point of our construction is an observation that there is a connection between bit-entry information retrieval and set intersection. Consider a database DB containing n entries $DB_1, \dots, DB_n$, where each entry DB_i is a bit. If we define the set $L = \{j \in [n] : DB_j = 1\}$, then for any $i \in [n]$ we have $DB_i = 1 \Leftrightarrow i \in L$. To retrieve the i -th bit entry in the database, the client just needs to know whether $i \in L$. Furthermore, assume that the client wants to retrieve τ entries with indexes $i_1, \dots, i_\tau$, then it is sufficient for the client to get all the retrieved entries that it knows the intersection $L \cap Q$, where $Q = \{i_1, \dots, i_\tau\}$. With our observation, we can obtain the following BatchSPIR protocol.

- 1) *Input.* The server takes an n -bit database $DB = (DB_1, \dots, DB_n)$ as input, and the client takes an index set $Q \subseteq [n]$ as input.
- 2) *Protocol.* The server defines the set $L = \{j \in [n] : DB_j = 1\}$. The server and the client run a PSI protocol Π where the server playing the role of the sender takes L as input and the client playing the role of the receiver takes Q as input. Let T be the output of the client.
- 3) *Output.* The client extracts the retrieved bits. Concretely, for each $j \in Q$, the retrieved bit is 1 if $j \in T$ and 0 otherwise.

The correctness of the above BatchSPIR protocol follows directly from the correctness of the underlying PSI protocol. However, we note that there exists a security flaw in the above protocol. In most existing PSI protocols, the sizes of the sets are public. However, in the above protocol, the size of the server's set will leak information about the database to the client. To deal with this, we let the server pad its set with $n - |L|$ dummy items (e.g., the items from $\{n + 1, \dots, 2n\}$), which makes the size of the new set be fixed to n . As we will show, if we modify the protocol in this way, then the resulting BatchSPIR protocol will be secure.

Instantiation with Unbalanced PSI. We instantiate our BatchSPIR based on the PSI protocol of [40] (with optimizations given by [42], [43]). Typically, the number of queries in (S)PIR is much smaller than the total number of entries in the database. This means that in our black-box BatchSPIR from PSI, the set of the receiver is much smaller than the set of the sender. This is exactly the unbalanced PSI setting considered by [29], [39]–[43]. We choose the unbalanced PSI protocol [40], [42], [43] and plug it in our black-box construction, as this protocol achieves sublinear communication in the size of the larger set. As a result, we obtain a BatchSPIR with sublinear communication cost.

As a side contribution, we also introduce new optimizations for the window technique used in [40], [42], [43], which enables better efficiency in many cases.

Concrete Implementation. We fully implemented BitBatSPIR based on the open-source implementation of the PSI protocol of [40], [42], [43]. We compared our protocol with VecBatchPIR [24], which is the state-of-the-art PIR protocol in the batch setting for databases with proper entry size (≤ 512 bytes). Our experimental results show that compared to VecBatchPIR, BitBatSPIR not only has a stronger security guarantee (symmetric privacy) but also achieves higher efficiency, especially for large databases.

D. Applications

Earlier, we briefly introduced two specific applications of PIR in the fields of drug purchase and advertising. Now, we make a further discussion. We will describe the applications of PIR from the perspective of privacy protection in the financial and medical fields. Our improved PIR protocol enables privacy preservation in these scenarios at a lower cost.

Applications in the Medical Field. In the medical field, patient privacy is of paramount importance. PIR technology can be applied to ensure that patients can access their medical records without revealing sensitive information about their health conditions. For instance, when a patient wants to retrieve specific medical records from a hospital's database, PIR allows them to do so without disclosing which particular records they are accessing. This is particularly useful in scenarios where patients may be concerned about the confidentiality of their medical history, such as in cases of sensitive conditions like mental health issues or sexually transmitted diseases.

Moreover, PIR can be utilized in medical research. Researchers often need to access large datasets of patient information to conduct studies, but this must be done without

compromising patient privacy. With PIR, researchers can query databases to retrieve aggregated data for analysis without learning the identities of the patients or specific details that could lead to re-identification. This enables advancements in medical research while maintaining strict privacy standards.

Applications in the Financial Field. In the financial sector, PIR offers significant benefits for protecting customer privacy and enhancing security. Customers can use PIR to check their account balances, transaction histories, or other financial information without revealing their account details or the specific nature of their inquiries to the financial institution. This is particularly valuable in preventing potential data breaches and identity theft, as sensitive information is not exposed during routine transactions.

PIR can also be applied in investment research and analysis. Investors often need to access market data and financial reports to make informed decisions. However, revealing their specific interests or strategies could potentially impact market dynamics or expose them to risks. By using PIR, investors can retrieve relevant financial information without disclosing their particular investment targets or strategies, thus maintaining a competitive edge and ensuring their privacy.

Furthermore, in the context of financial advisory services, PIR can enable clients to seek advice and access personalized financial planning tools without compromising their financial status or goals. This allows for a more secure and private interaction between clients and financial advisors, fostering trust and ensuring compliance with data protection regulations.

II. PRELIMINARIES

A. Basic Notations

Let κ and λ be the computational and statistical security parameters, respectively. For any integers m and n , we denote $[m, n]$ the set $\{m, \dots, n\}$ if $m \leq n$ and the empty set $\emptyset$ if $m > n$. For the special case of $m = 1$, we abbreviate $[1, n]$ by $[n]$. For any set X , let $|X|$ be the cardinality of X . We say a function $\varepsilon(\kappa)$ is negligible, denoted $\varepsilon(\kappa) = \text{negl}(\kappa)$, if for every positive integer c there exists an integer n_c such that for all $x > n_c$, it holds that $\varepsilon(x) < 1/x^c$. For any two distributions D_0, D_1 , we say D_0 and D_1 are computationally indistinguishable, denoted $D_0 \approx_c D_1$, if no probabilistic polynomial-time (PPT) algorithm can distinguish them. For any plaintext x , we use $\llbracket x \rrbracket$ to denote an encryption of x .

B. Adversarial Model

We construct our protocol under the semi-honest model, which means that the adversary can only obtain the internal state of the corrupted party, but cannot cause the corrupted party to deviate from the execution of the protocol. We note that this is meaningful in scenarios where all parties will follow the protocol specification, and the adversary wants to attack the protocol without changing the code of some parties. In this way, the parties cannot detect if they are being attacked by observing whether their code changes.

C. Primitive Definitions

In this section, we give the formal definitions of BatchSPIR and PSI.

Definition II.1 (Batch Symmetric Private Information Retrieval). *Batch Symmetric Private Information Retrieval (BatchSPIR) is a two-party protocol, where the server takes a database $DB = \{DB_i\}_{i \in [n]}$ containing n entries as input and the client takes an index set $Q \subseteq [n]$ as input. BatchSPIR has the following requirements.*

- *Correctness.* At the end of the protocol, the client obtains the retrieved entries $\{DB_j\}_{j \in Q}$.
- *Client-Privacy.* The server knows nothing about Q . Concretely, if we let $\text{View}_{\text{ser}}^{\text{spir}}(DB, Q)$ be the view of the server, then there exists a PPT algorithm $\text{Sim}_{\text{ser}}^{\text{spir}}$ taking DB as input such that

$$\text{View}_{\text{ser}}^{\text{spir}}(DB, Q) \approx_c \text{Sim}_{\text{ser}}^{\text{spir}}(DB).$$

- *Server-Privacy.* The client knows nothing about DB except $\{DB_j\}_{j \in Q}$. Concretely, let $\text{View}_{\text{cli}}^{\text{spir}}(DB, Q)$ be the view of the client, then there exists a PPT algorithm $\text{Sim}_{\text{cli}}^{\text{spir}}$ taking $Q, \{DB_j\}_{j \in Q}$ as inputs such that

$$\text{View}_{\text{cli}}^{\text{spir}}(DB, Q) \approx_c \text{Sim}_{\text{cli}}^{\text{spir}}(Q, \{DB_j\}_{j \in Q}).$$

Definition II.2 (Private Set Intersection). *Private Set Intersection (PSI) is a two-party protocol where one party called the sender takes a private set $X_0 \subseteq \{0, 1\}^l$ as input and the other party called the receiver takes a private set $X_1 \subseteq \{0, 1\}^l$ as input. PSI has the following requirements.*

- *Correctness.* At the end of the protocol, the receiver obtains the intersection $X_0 \cap X_1$.
- *Receiver-Privacy.* The sender cannot know anything about X_1 . Concretely, let $\text{View}_{\text{sen}}^{\text{psi}}(X_0, X_1)$ be the view of the sender, then there exists a PPT algorithm $\text{Sim}_{\text{sen}}^{\text{psi}}$ taking X_0 as input such that

$$\text{View}_{\text{sen}}^{\text{psi}}(X_0, X_1) \approx_c \text{Sim}_{\text{sen}}^{\text{psi}}(X_0).$$

- *Sender-Privacy.* The receiver knows nothing about X_0 except $X_0 \cap X_1$. Concretely, let $\text{View}_{\text{rec}}^{\text{psi}}(X_0, X_1)$ be the view of the receiver, then there exists a PPT algorithm $\text{Sim}_{\text{rec}}^{\text{psi}}$ taking $X_0 \cap X_1$ and X_1 as inputs such that

$$\text{View}_{\text{rec}}^{\text{psi}}(X_0, X_1) \approx_c \text{Sim}_{\text{rec}}^{\text{psi}}(X_0 \cap X_1, X_1).$$

We denote (N_0, N_1, l) -PSI a PSI protocol where the elements are l -bit long in each set and the set size of the sender (resp. receiver) is N_0 (resp. N_1).

D. Leveled Fully Homomorphic Encryption

Fully Homomorphic Encryption (FHE) is a powerful primitive that allows one to directly operate on encrypted data without knowing the decryption key. The first FHE scheme was given by Gentry [45] in 2009. This scheme is impractical due to the use of the expensive bootstrapping operations which have a large computational cost. Fortunately, much more efficient FHE schemes were constructed in [46]–[50], and particularly, Leveled FHE schemes like BGV [46] and BFV [47], [51] avoid the use of the expensive bootstrapping

operations and are efficient enough for computing low-depth circuits.

Like the previous PIR and SPIR protocols, our protocol also uses FHE. Since Leveled FHE avoids the expensive bootstrapping operations, we will use Leveled FHE in our protocols. The two main Leveled FHE schemes BGV [46] and BFV [47], [51] share the same plaintext and ciphertext space. More precisely, both of these two schemes are defined in a cyclotomic ring $\mathcal{R} = \mathbb{Z}[x]/(x^N + 1)$ with N being some integer. The plaintext space is $\mathcal{R}_t = \mathcal{R}/(t\mathcal{R})$ for some prime number t . The integer t is called the plaintext modulus. The ciphertext space is $\mathcal{R}_q^2$ where q is an integer larger than t and $\mathcal{R}_q = \mathcal{R}/(q\mathcal{R})$. The integer q is called the ciphertext modulus. Moreover, the plaintext length is $N \log t$ and the ciphertext length is $2N \log q$.

In all existing FHE schemes, homomorphic operations will introduce noise to the operated ciphertext, and the amount of noise in the ciphertext will influence the probability of successful decryption. For this reason, the cost of a homomorphic operation depends on the computational cost and the amount of noise it introduces. It is well-known that homomorphic non-scalar multiplication is the most costly homomorphic operation in both the running time and noise growth (we refer to [52], [53] for more details about homomorphic operations and their costs). For this reason, minimizing the number of non-scalar multiplications and the multiplicative depth is of great importance for reducing the total computational and communication cost.

A Note on the Security of FHE. In this work, we require the leveled FHE scheme is not only IND-CPA secure, but also achieves circuit privacy, which implies that the distribution of the outputs of some homomorphic operations is indistinguishable from that of fresh encryptions of the plaintext outputs. We remark that when using OPRF in our protocol, we no longer need circuit privacy, at the cost of a higher offline computation (see Section V-A for more details).

III. BLACK-BOX CONSTRUCTION OF BATCHSPIR FROM PSI

In this section, we present our black-box construction of BatchSPIR from PSI. Recall that our focus is that the entries in the database are bits. The extension to the setting that the entries are bitstrings is direct using the idea of [7], and we will discuss this in Section IV-C. But we note that we will estimate the efficiency of our protocol in the case of the entries being bits, while in the case of the entries being bitstring, the efficiency of our protocol seems to be unsatisfactory, and we leave it to future work to optimize our protocol for databases with long entries.

Our BatchSPIR construction requires one call to a PSI protocol. Protocol $\Pi_{\text{batchspir}}$ is the formal description of our black-box construction of BatchSPIR from PSI.

Protocol $\Pi_{\text{batchspir}}$. BatchSPIR from PSI

Input: The server $\mathcal{S}$ takes a database $DB = (DB_1, \dots, DB_n) \in \{0, 1\}^n$ as input and the client $\mathcal{C}$

takes an index set $Q \subseteq [n]$ with $|Q| = \tau$ as input.
Output: The client outputs $\{DB_j\}_{j \in Q}$.

- 1) $\mathcal{S}$ defines the set $L = \{j \in [n] : DB_j = 1\}$ and chooses some set $D \subseteq [n+1, 2n]$ of size $n - |L|$. $\mathcal{S}$ defines the set $P = L \cup D$.
- 2) $\mathcal{S}$ and $\mathcal{C}$ invoke a $(n, \tau, \lceil \log 2n \rceil)$ -PSI protocol Π_{psi} where $\mathcal{S}$ playing the role of the sender takes P as input and $\mathcal{C}$ playing the role of the receiver takes Q as input. Let T be the output of $\mathcal{C}$.
- 3) For each $j \in Q$, $\mathcal{C}$ sets u_j to 1 if $j \in T$ and 0 otherwise. Finally, $\mathcal{C}$ outputs $\{u_j\}_{j \in Q}$.

We state the security of $\Pi_{\text{batchspir}}$ by proving the following theorem.

Theorem III.1. *If Π_{psi} is a secure PSI protocol, then $\Pi_{\text{batchspir}}$ is a secure BatchSPIR protocol.*

Proof. To prove that $\Pi_{\text{batchspir}}$ is a secure BatchSPIR protocol, we need to show the correctness, client-privacy, and server-privacy defined in Definition II.1.

Correctness. We need to show that for each $j \in Q$, it holds that $u_j = DB_j$. If $j \in T$, then we have $u_j = 1$. On the other hand, by the correctness of Π_{psi} , we have $T = P \cap Q = L \cap Q$, which implies that $j \in L$. Therefore, by the definition of L , we know that $DB_j = 1 = u_j$.

If $j \notin T$, then we have $u_j = 0$. Moreover, since $j \in Q, j \notin T$, and $T = P \cap Q = L \cap Q$, we know that $j \notin L$. Therefore, by the definition of L , we know that $DB_j = 0 = u_j$.

Client-Privacy. Let $\text{View}_{\text{ser}}(\text{DB}, Q)$ be the view of the server, then we need to construct a simulator Sim taking DB as input such that

$$\text{View}_{\text{ser}}(\text{DB}, Q) \approx_c \text{Sim}(\text{DB}).$$

By the receiver-privacy of Π_{psi} , there exists a PPT algorithm Sim_{psi} taking P as input such that

$$\text{View}_{\text{ser}}(\text{DB}, Q) \approx_c \text{Sim}_{\text{psi}}(P).$$

Now the simulator Sim works as follows.

- 1) Invoke the code of the server to define the sets L, D, P as in the protocol.
- 2) Invoke the algorithm Sim_{psi} on P , and let z be the output.
- 3) Return z .

It is easy to see that the distribution of z is identical to $\text{Sim}_{\text{psi}}(P)$. Therefore, we have $\text{Sim}(\text{DB}) \approx_c \text{View}_{\text{ser}}(\text{DB}, Q)$.

Server-Privacy. Let $\text{View}_{\text{cli}}(\text{DB}, Q)$ be the view of the client, then we need to construct a simulator Sim taking $Q, \{u_j\}_{j \in Q}$ as inputs such that

$$\text{View}_{\text{cli}}(\text{DB}, Q) \approx_c \text{Sim}(Q, \{u_j\}_{j \in Q}).$$

By the sender-privacy of Π_{psi} , there exists a PPT algorithm Sim_{psi} taking Q, T as inputs such that

$$\text{View}_{\text{cli}}(\text{DB}, Q) \approx_c \text{Sim}_{\text{psi}}(Q, T).$$

Now the simulator Sim works as follows.

- 1) Define the set $T' = \{j \in Q : u_j = 1\}$.
- 2) Invoke the algorithm Sim_{psi} on (Q, T') and let z be the output.
- 3) Return z .

By the definition of T' , we know $T' = T$, which implies the output z is identical to $\text{Sim}_{\text{psi}}(Q, T)$. Therefore, we have $\text{Sim}(Q, \{u_j\}_{j \in Q}) \approx_c \text{View}_{\text{cli}}(\text{DB}, Q)$. $\square$

IV. OUR BATCHSPIR PROTOCOL

In this section, we introduce **BitBatSPIR**, an efficient BatchSPIR protocol. Our protocol is derived by applying the (Leveled) FHE-based PSI protocol of [40], [42], [43] to our black-box construction of BatchSPIR. The works of [40], [42], [43] first presented a basic PSI protocol and then optimized this basic PSI protocol to obtain a full PSI protocol with better efficiency. Similarly, we first present our basic BatchSPIR protocol which is based on their basic PSI protocol. Let $(\text{KeyGen}, \text{Enc}, \text{Dec})$ be a leveled FHE scheme, and we assume that the client has generated a key pair $(pk, sk) \leftarrow \text{KeyGen}(1^\kappa)$ and sent the public key pk to the server. Now we give the formal description of our basic BatchSPIR protocol.

Protocol Π_{basic} . The Basic BatchSPIR Protocol

Input: The server $\mathcal{S}$ takes a database $\text{DB} = (DB_1, \dots, DB_n) \in \{0, 1\}^n$ as input and the client $\mathcal{C}$ takes an index set $Q \subseteq [n]$ with $|Q| = \tau$ as input.
Output: The client $\mathcal{C}$ outputs $\{DB_j\}_{j \in Q}$.

- 1) *Query.* Assume $Q = \{i_1, \dots, i_\tau\}$, then for each $j \in [\tau]$, $\mathcal{C}$ computes $c_j \leftarrow \text{Enc}(pk, i_j)$. Then, $\mathcal{C}$ sends $\{c_j\}_{j \in [\tau]}$ to $\mathcal{S}$.
- 2) *Response.* $\mathcal{S}$ does the followings.
 - a) Define the set $L = \{j \in [n] : DB_j = 1\}$ and chooses some set $D \subseteq [n+1, 2n]$ of size $n - |L|$.
 - b) Define the set $P = L \cup D$ and the polynomial $f(x) = \prod_{j \in P} (x - j)$.
 - c) Sample τ non-zero random plaintext $r_1, \dots, r_\tau$.
 - d) For each $j \in [\tau]$, compute an encryption e_j of $r_j f(i_j)$ using the ciphertext c_j (of i_j).
 - e) Send $\{e_j\}_{j \in [\tau]}$ to $\mathcal{C}$.
- 3) *Extract.* $\mathcal{C}$ extracts the retrieved entries. Concretely, for each $j \in [\tau]$, $\mathcal{C}$ decrypts $z_j \leftarrow \text{Dec}(sk, e_j)$ and sets u_{i_j} to 1 if $z_j = 0$ and 0 otherwise. $\mathcal{C}$ outputs $\{u_i\}_{i \in Q}$.

The correctness of the basic BatchSPIR protocol Π_{basic} is easy to verify, which follows from that for each $i \in [n]$ it holds that

$$DB_i = 1 \Leftrightarrow i \in P \Leftrightarrow f(i) = 0.$$

The security is based on the IND-CPA security and circuit privacy of the underlying FHE scheme. The IND-CPA security guarantees that the server knows nothing about Q from the

ciphertexts $\{c_j\}_{j \in [\tau]}$, and the circuit privacy guarantees that the client only gets ciphertexts that are indistinguishable from fresh encryptions of $\{r_j f(i_j)\}_{j \in [\tau]}$, which only leaks $\{\text{DB}_i\}_{i \in Q}$ because each r_j is a non-zero random plaintext.

Although the basic BatchSPIR protocol achieves a low communication cost, it suffers from a high computational cost, which makes the protocol impractical in many settings. Concretely, it requires the server to compute a depth- $O(\log n)$ circuit with $O(\tau n)$ homomorphic non-scalar multiplications. This will result in a relatively high computational cost for typical choices of n (e.g., $2^{20}, 2^{30}$). Fortunately, the works of [40], [42], [43] introduced a set of techniques to reduce the computational cost. With these techniques, we obtain our final BatchSPIR protocol BitBatSPIR, which achieves high efficiency in both the asymptotic and concrete aspects. In the remainder of this section, we describe some of these techniques which significantly reduces the asymptotic cost. With these two techniques, the server only needs to compute a depth- $O(\log \log n)$ circuit using $O(\tau \sqrt{n})$ homomorphic non-scalar multiplications. We refer the reader to [43, Section 3] for several other techniques which further reduces the concrete cost and enables more tradeoffs.

Correction and Concrete Improvements. As a side contribution, we optimize one of the techniques that we will describe below (i.e., the windowing technique). Concretely, we fix a flaw in the analysis given by [43] and present the corresponding result. Moreover, we find that though their analysis gives us a good asymptotic communication cost, they also introduce a lower bound on the concrete communication cost, thus prevent us from achieving lower communication. Our optimization allows us to break this lower bound.

A. Reducing the Number of Non-Scalar Multiplications

The work [43] showed how to use the Paterson-Stockmeyer algorithm [60] to reduce the number of non-scalar multiplications. We review this technique. Note that in our basic protocol, the server needs to homomorphically compute τ degree- n polynomials, where each polynomial is of form $f(x) = \prod_{j \in B} (x - j)$ and $|B| = n$. A naive method for computing $f(x)$ will consume $n - 1$ non-scalar multiplications¹. However, if we write $f(x)$ as $f(x) = \sum_{j=0}^n a_j x^j$,² then it is possible to compute $f(x)$ using only $O(\sqrt{n})$ non-scalar multiplications. Concretely, note that for any $\alpha \in [n + 1]$ and $\beta = \lceil (n + 1)/\alpha \rceil$, the polynomial $f(x)$ can be written as

$$f(x) = \sum_{j=0}^{\beta-2} x^{j\alpha} \sum_{k=0}^{\alpha-1} a_{j\alpha+k} x^k + x^{(\beta-1)\alpha} \sum_{j=0}^{n-(\beta-1)\alpha} a_{j+(\beta-1)\alpha} x^j.$$

Based on this equation, $f(x)$ can be computed in the following steps.

- 1) Compute $\mu_i = x^i$ for all $i \in [2, \alpha]$ and $\eta_j = x^{j\alpha}$ for all $j \in [2, \beta - 1]$. Set $\eta_0 = 1$ and $\eta_1 = \mu_\alpha$.

¹Since homomorphic non-scalar multiplication is the most costly homomorphic operation, hence minimizing the number of homomorphic non-scalar multiplications is of great importance for reducing the computational cost.

²The coefficients $\{a_j\}_{j \in [0, n]}$ can be precomputed by the server locally.

- 2) For each $j \in [0, \beta - 2]$, compute $\delta_j = a_{j\alpha} + \sum_{k=1}^{\alpha-1} a_{j\alpha+k} \mu_k$.
- 3) Compute $\delta_{\beta-1} = \sum_{k=1}^{n-(\beta-1)\alpha} a_{(\beta-1)\alpha+k} \mu_k + a_{(\beta-1)\alpha}$.
- 4) Compute $f(x) = \delta_0 + \sum_{j=1}^{\beta-1} \eta_j \delta_j$.

The required number of non-scalar multiplications is

$$\underbrace{(\alpha - 1 + \beta - 2)}_{\text{step 1}} + \underbrace{(\beta - 1)}_{\text{step 4}} = \alpha + 2\beta - 4.$$

If we choose $\alpha = \Theta(\sqrt{n})$, then we know that $O(\sqrt{n})$ non-scalar multiplications are sufficient to compute $f(x)$. As a result, the total number of non-scalar multiplications of our protocol will be $\tau \cdot O(\sqrt{n}) = O(\tau \sqrt{n})$.

In practice, to minimize the number of non-scalar multiplications, we choose α to minimize $\alpha + 2\beta - 4$ where $\beta = \lceil (n + 1)/\alpha \rceil$. We argue that the minimum value of $\alpha + 2\beta - 4$ is upper-bounded by $2\sqrt{2} \cdot \sqrt{n + 1} - 1$. This comes from that if we choose $\alpha = \lceil \sqrt{2n + 2} \rceil$, then we have

$$\begin{aligned} \alpha + 2\beta - 4 &= \alpha + 2 \lceil \frac{n+1}{\alpha} \rceil - 4 \\ &< \alpha + 2 \left(\frac{n+1}{\alpha} + 1 \right) - 4 \\ &= \alpha + \frac{2n+2}{\alpha} - 2 \\ &= \lceil \sqrt{2n+2} \rceil + \frac{2n+2}{\lceil \sqrt{2n+2} \rceil} - 2 \\ &< \sqrt{2n+2} + 1 + \frac{2n+2}{\sqrt{2n+2}} - 2 \\ &= 2\sqrt{2} \cdot \sqrt{n+1} - 1. \end{aligned}$$

Partitioning. Using the partitioning technique introduced by [40], we can reduce the depth and the number of non-scalar multiplications at the price of more communication. Recall that the server needs to homomorphically compute τ polynomials, where each polynomial is of form $f(x) = \prod_{j \in B} (x - j)$ for some set B of size n . The idea of partitioning is that we split the set B into s parts $\{B_k\}_{k \in [s]}$ for some $s \in [n]$, where each part is of size n_0 or $n_0 - 1$ with $n_0 = \lceil n/s \rceil$.³ Instead of computing $f(x)$, the server computes the s polynomials $\{f_k(x)\}_{k \in [s]}$, where each $f_k(x) = \prod_{j \in B_k} (x - j)$. Then the server sends the resulting s ciphertexts to the client. Finally, the client sets the retrieved bit to 1 if and only if at least one of the s ciphertexts is decrypted to 0.

The above method allows us to reduce the depth and the number of non-scalar multiplications at the price of more communication (there are s ciphertexts that must be sent by the server). On one hand, the server only needs to compute polynomials of degree n_0 instead of n , which allows us to reduce the depth. On the other hand, if the server uses the Paterson-Stockmeyer algorithm, then the powers of x can be reused for each partition, thus reducing the number of non-scalar multiplications. Concretely, let $\alpha \in [n_0 + 1]$ be the parameter used in the Paterson-Stockmeyer algorithm, then the computation of the powers $\{x^j\}_{j \in [2, \alpha]}$ and $\{x^{j\alpha}\}_{j \in [2, \beta-1]}$ (where $\beta = \lceil (n_0 + 1)/\alpha \rceil$) requires $\alpha + \beta - 3$ non-scalar

³More precisely, there are $s \lceil n/s \rceil - n$ parts which are of size $\lceil n/s \rceil - 1$ and the remaining parts are of size $\lceil n/s \rceil$.

TABLE I

USING THE PATERSON-STOCKMEYER ALGORITHM, THE MINIMUM VALUE OF THE NUMBER OF NON-SCALAR MULTIPLICATIONS IS COMPUTED BY FINDING THE MINIMUM VALUE IN THE SET $\{\alpha + (s+1)\beta - s - 3\}_{\alpha \in [m_0+1]}$ WHERE $m_0 = \lceil m/s \rceil$ AND $\beta = \lceil (m_0+1)/\alpha \rceil$. NOTE THAT $s = 1$ MEANS THAT WE DO NOT USE THE PARTITIONING TECHNIQUE.

$m \backslash s$	1	2	2^2	2^3	2^4	2^5	2^6
2^{20}	2891	2501	2278	2153	2076	2013	1933
2^{25}	16379	14181	12941	12269	11906	11697	11544
2^{30}	92676	80257	73260	69492	67517	66485	65915

multiplications. These powers of x can be reused for each partition, hence the total number of non-scalar multiplications is $\alpha + \beta - 3 + s(\beta - 1) = \alpha + (s+1)\beta - s - 3$. To verify that we can reduce the number of non-scalar multiplications, we present Table I, which shows the minimum value of the number of non-scalar multiplications for some given pairs of (n, s) . For example, by setting $s = 8$, Table I tells us that we can reduce the number of non-scalar multiplications by around 1/4.

B. Windowing: Reducing the Depth

Using the Paterson-Stockmeyer algorithm allows us to reduce the number of non-scalar multiplications from $O(n)$ to $O(\sqrt{n})$. However, the server still needs to compute a depth- $(d_0 + d_1 + 1)$ circuit where $d_0 = \lceil \log_2 \alpha \rceil$, $d_1 = \lceil \log_2(\beta - 1) \rceil$. Note that

$$d_0 + d_1 + 1 = \lceil \log_2 \alpha \rceil + \lceil \log_2(\beta - 1) \rceil + 1 > \log(\max(\alpha - 1, \beta - 1)).$$

Since $\beta = \lceil (n+1)/\alpha \rceil \geq (n+1)/\alpha$, we have $\max(\alpha, \beta) \geq \sqrt{n+1}$. Therefore, we know that

$$d_0 + d_1 + 1 > \log(\max(\alpha - 1, \beta - 1)) = \Omega(\log n).$$

In this section, we show how to use the windowing technique [40], [43] to reduce the depth to $O(\log \log n)$. As an independent contribution, we fix a flaw in their analysis and present an optimization. Our optimization enables more trade-offs between communication and computation. Concretely, we find that their analysis introduces a lower bound on the concrete communication cost. We will show how to break this lower bound.

Windowing. Using the Paterson-Stockmeyer algorithm, the server needs to compute encryptions of the powers $\{x^j\}_{j \in [2, \alpha]}$ and $\{x^{j\alpha}\}_{j \in [2, \beta-1]}$, where $\alpha = \Theta(\sqrt{n})$ and $\beta = \lceil (n+1)/\alpha \rceil$. The computation of these powers requires the server to compute a circuit with depth $\Omega(\log n)$. However, if the server has more powers of x , the depth can be reduced. For example, if the server has all of these powers, then it only needs to compute the polynomial using a circuit with depth 1. The work of [43] proved a lemma (see [43, Lemma 1]), which gives an upper-bound on the number of powers for computing a polynomial $f(x)$ of degree $n > 0$ using a circuit with depth $d \geq 0$. However, there is a flaw in their proof, which slightly influences their result. We first correct their proof and give the

corresponding result using a similar analysis. The corrected lemma is as follows.

Lemma IV.1. *To compute a polynomial $f(x)$ of degree $n > 0$ with $O(\sqrt{n})$ non-scalar multiplications and multiplicative depth $d \geq 3$, the server requires less than $2^{d-1}(n+1)^{(2^{d-1}-2)^{-1}}$ powers.⁴ If we further require that the number of non-scalar multiplications is $2\sqrt{2n} + O(1)$, then the number of the required powers is less than $3\sqrt{2} \cdot 2^{d-3}(n+1)^{(2^{d-1}-2)^{-1}}$.*

Proof. To compute a degree- n polynomial using $O(\sqrt{n})$ non-scalar multiplications, we use the Paterson-Stockmeyer algorithm with parameters $\alpha = \Theta(\sqrt{n})$ and $\beta = \lceil (n+1)/\alpha \rceil$. The server needs to compute the powers $\{x^j\}_{j \in [\alpha-1]}$ and $\{x^{j\alpha}\}_{j \in [\beta-1]}$. Note that computing $f(x)$ using these powers only requires computing a depth-1 circuit, hence the target depth for computing these powers is $d' = d - 1 \geq 2$.

We first consider the computation of the powers $\{x^j\}_{j \in [\alpha-1]}$. Note that for any $b_\alpha \in [2, \alpha - 1]$, we can decompose $\alpha - 1$ in base b_α as

$$\alpha - 1 = \sum_{k=0}^{\lceil \log_{b_\alpha}(\alpha-1) \rceil} \alpha_k b_\alpha^k$$

with $\alpha_k \in [0, b_\alpha - 1]$. If the client sends the server the encryptions of all the powers $\{x^{j b_\alpha^k}\}_{j \in [b_\alpha-1], k \in [0, \lceil \log_{b_\alpha}(\alpha-1) \rceil]}$, then the server can compute the power $x^{\alpha-1}$ using a circuit with depth $\lceil \log_2(\lceil \log_{b_\alpha}(\alpha-1) \rceil + 1) \rceil$. Moreover, the server can compute all the powers $\{x^j\}_{j \in [\alpha-1]}$ using a circuit with the same depth. Our goal is to minimize the number $(b_\alpha - 1)(\lceil \log_{b_\alpha}(\alpha-1) \rceil + 1)$ under the condition that the depth $\lceil \log_2(\lceil \log_{b_\alpha}(\alpha-1) \rceil + 1) \rceil$ is no larger than d' . Note that

$$\lceil \log_2(\lceil \log_{b_\alpha}(\alpha-1) \rceil + 1) \rceil < \log_2(\log_{b_\alpha}(\alpha-1) + 1) + 1.$$

This implies that $\log_2(\log_{b_\alpha}(\alpha-1) + 1) + 1 \leq d'$ is sufficient to guarantee that $\lceil \log_2(\lceil \log_{b_\alpha}(\alpha-1) \rceil + 1) \rceil \leq d'$.⁵ We choose b_α to be $\lceil (\alpha-1)^{(2^{d'-1}-1)^{-1}} \rceil$, the smallest integer satisfying $\log_2(\log_{b_\alpha}(\alpha-1) + 1) + 1 \leq d'$, then we have

$$\begin{aligned} & (b_\alpha - 1)(\lceil \log_{b_\alpha}(\alpha-1) \rceil + 1) \\ & \leq (b_\alpha - 1)(\log_{b_\alpha}(\alpha-1) + 1) \\ & < (\alpha-1)^{(2^{d'-1}-1)^{-1}} \left(\log_{(\alpha-1)^{(2^{d'-1}-1)^{-1}}}(\alpha-1) + 1 \right) \\ & = 2^{d'-1}(\alpha-1)^{(2^{d'-1}-1)^{-1}}. \end{aligned}$$

This implies that in order to enable the server to compute the powers $\{x^j\}_{j \in [\alpha-1]}$ using a circuit with depth within d' , the client only needs to send less than $2^{d'-1}(\alpha-1)^{(2^{d'-1}-1)^{-1}}$ ciphertexts. By a similar discussion, the client needs to send less than $2^{d'-1}(\beta-1)^{(2^{d'-1}-1)^{-1}}$ ciphertexts to enable the server to compute the powers $\{x^{j\alpha}\}_{j \in [\beta-1]}$ using a circuit with

⁴The original (wrong) bound given by [43] is $3(n+1)^{2^{-d}}(2^{d-1}+1)$ for any $d \geq 0$.

⁵The work [43] required that $\log_2 \log_{b_\alpha}(\alpha-1)$, a lower bound of $\lceil \log_2(\lceil \log_{b_\alpha}(\alpha-1) \rceil + 1) \rceil$, is no more than d' . This cannot guarantee that $\lceil \log_2(\lceil \log_{b_\alpha}(\alpha-1) \rceil + 1) \rceil \leq d'$. To correct their analysis, we use an upper bound instead of a lower bound.

depth within d' . Therefore, the total number of ciphertexts is less than

$$2^{d'-1} \left((\alpha - 1)^{(2^{d'-1}-1)^{-1}} + (\beta - 1)^{(2^{d'-1}-1)^{-1}} \right).$$

Note that to ensure that the number of non-scalar multiplications is $O(\sqrt{n})$, we require that $\alpha = \Theta(\sqrt{n})$ (and $\beta = \lceil (n+1)/\alpha \rceil$). If we choose $\alpha = \lceil \sqrt{n+1} \rceil$, then we have

$$\begin{aligned} & 2^{d'-1} \left((\alpha - 1)^{(2^{d'-1}-1)^{-1}} + (\beta - 1)^{(2^{d'-1}-1)^{-1}} \right) \\ & < 2^{d'-1} \left((\sqrt{n+1})^{(2^{d'-1}-1)^{-1}} + \left(\frac{n+1}{\sqrt{n+1}} \right)^{(2^{d'-1}-1)^{-1}} \right) \\ & = 2^{d'} (n+1)^{(2^{d'-2})^{-1}}. \end{aligned}$$

We note that $\alpha = \lceil \sqrt{n+1} \rceil$ will result in $3\sqrt{n} + O(1)$ non-scalar multiplications. If we want the number of non-scalar multiplications to be $2\sqrt{2n} + O(1)$, then we can choose $\alpha = \lceil \sqrt{2n+2} \rceil$ ⁶. This way, the number of the ciphertexts sent by the client satisfies that

$$\begin{aligned} & 2^{d'-1} \left((\alpha - 1)^{(2^{d'-1}-1)^{-1}} + (\beta - 1)^{(2^{d'-1}-1)^{-1}} \right) \\ & < 2^{d'-1} \left((\sqrt{2n+2})^{(2^{d'-1}-1)^{-1}} + \left(\frac{n+1}{\sqrt{2n+2}} \right)^{(2^{d'-1}-1)^{-1}} \right) \\ & = 2^{d'-1} \left(2^{(2^{d'-2})^{-1}} + 2^{-(2^{d'-2})^{-1}} \right) (n+1)^{(2^{d'-2})^{-1}} \\ & \leq 3\sqrt{2} \cdot 2^{d'-2} (n+1)^{(2^{d'-2})^{-1}}, \end{aligned}$$

where the last inequality follows from that $d' = d-1 \geq 2$. $\square$

By setting $d = \Theta(\log \log n)$, Lemma IV.1 tells us that we can reduce the depth to $O(\log \log n)$ at the price that the client sends $O(\log n)$ ciphertexts. Lemma IV.1 gives us a good asymptotic complexity of the number of required ciphertexts. However, we argue that the method used in the proof of Lemma IV.1 also introduces a lower bound on the number of required ciphertexts, as we discussed below.

Limitation of the Windowing Technique Used in [43]. Recall that in the proof of Lemma IV.1, the client must send

$$(b_\alpha - 1)(\lfloor \log_{b_\alpha}(\alpha - 1) \rfloor + 1) + (b_\beta - 1)(\lfloor \log_{b_\beta}(\beta - 1) \rfloor + 1)$$

encryptions where $b_\alpha \in [2, \alpha - 1]$ and $b_\beta \in [2, \beta - 1]$. We use C to denote this number. Now we show that C has a lower bound. Concretely, note that

$$\begin{aligned} C &= (b_\alpha - 1)\lceil \log_{b_\alpha} \alpha \rceil + (b_\beta - 1)\lceil \log_{b_\beta} \beta \rceil \\ &\geq (b_\alpha - 1)\log_{b_\alpha} \alpha + (b_\beta - 1)\log_{b_\beta} \beta \\ &= \frac{b_\alpha - 1}{\log_2 b_\alpha} \log_2 \alpha + \frac{b_\beta - 1}{\log_2 b_\beta} \log_2 \beta, \end{aligned}$$

where the first equation follows from that for any integers $j \geq 2$ and $k \geq 1$, it holds that $\lfloor \log_j(k-1) \rfloor = \lceil \log_j k \rceil - 1$. Since for any $j \geq 2$, it holds that $(j-1)/\log_2 j \geq 1$, we have

$$C \geq \log_2 \alpha + \log_2 \beta = \log_2 \left(\alpha \lceil \frac{n+1}{\alpha} \rceil \right) \geq \log_2 (n+1).$$

⁶Recall that in Section IV-A, we have shown that if $\alpha = \lceil \sqrt{2n+2} \rceil$, then the number of non-scalar multiplications is less than $2\sqrt{2} \cdot \sqrt{n+1} - 1 = 2\sqrt{2n} + O(1)$.

TABLE II

EACH PAIR (j, k) REPRESENTS THAT THE NUMBER OF THE REQUIRED ENCRYPTED POWERS USING THE WINDOWING TECHNIQUE IN [43] AND THE OPTIMIZED WINDOWING TECHNIQUE ARE j AND k , RESPECTIVELY.

$m \backslash d'$	4	5	6	7	8	9	10
2^{20}	(21,12)	(21,7)	(21,5)	(21,4)	(21,4)	(21,4)	(21,4)
2^{25}	(26,21)	(26,9)	(26,6)	(26,5)	(26,4)	(26,4)	(26,4)
2^{30}	(31,31)	(31,12)	(31,8)	(31,6)	(31,6)	(31,4)	(31,4)

This means that using the windowing technique used in [43], the client must send at least $\lceil \log_2(n+1) \rceil$ encryptions. Now we present an optimization for the windowing technique. The optimized windowing technique allows us to break the aforementioned lower bound.

Breaking the Lower Bound. Let us first consider the computation of the powers $\{x^j\}_{j \in [\alpha-1]}$. Let $b_\alpha \in [2, \alpha - 1]$ be the base. Instead of sending the encryptions of all the powers $\{x^{jb_\alpha^k}\}_{j \in [b_\alpha-1], k \in [0, \lfloor \log_{b_\alpha}(\alpha-1) \rfloor]}$, we let the client send the encryptions of the basic powers $\{x^{b_\alpha^k}\}_{k \in [0, \lfloor \log_{b_\alpha}(\alpha-1) \rfloor]}$. We decompose $\alpha - 1$ in base b_α as

$$\alpha - 1 = \sum_{k=0}^{\lfloor \log_{b_\alpha}(\alpha-1) \rfloor} \alpha_k b_\alpha^k.$$

This implies that for each $j \in [\alpha - 1]$, the power x^j is the product of at most $(b_\alpha - 1)(\lfloor \log_{b_\alpha}(\alpha - 1) \rfloor + 1)$ basic powers. Therefore, the server can compute all the powers $\{x^j\}_{j \in [\alpha-1]}$ using a circuit with depth $\lceil \log_2((b_\alpha - 1)(\lfloor \log_{b_\alpha}(\alpha - 1) \rfloor + 1)) \rceil$. By a similar discussion, for any base $b_\beta \in [2, \beta - 1]$, the server can compute the powers $\{x^{j\alpha}\}_{j \in [\beta-1]}$ using a circuit with depth $\lceil \log_2((b_\beta - 1)(\lfloor \log_{b_\beta}(\beta - 1) \rfloor + 1)) \rceil$ if the client sends $\lfloor \log_{b_\beta}(\beta - 1) \rfloor + 1$ encrypted powers. Recall that d' is the target depth for computing the powers. Our goal is to minimize the number $\lfloor \log_{b_\alpha}(\alpha - 1) \rfloor + \lfloor \log_{b_\beta}(\beta - 1) \rfloor + 2$ under the condition that

$$\lceil \log_2((b_\alpha - 1)(\lfloor \log_{b_\alpha}(\alpha - 1) \rfloor + 1)) \rceil \leq d'$$

and

$$\lceil \log_2((b_\beta - 1)(\lfloor \log_{b_\beta}(\beta - 1) \rfloor + 1)) \rceil \leq d'.$$

To show that our optimized windowing technique allows us to reduce the number of powers, we present Table II, which shows the number of the required powers sent by the client for computing a degree- n polynomial using a depth- $(d' + 1)$ circuit for fixed values of n, d' . To derive this table, for a given (n, d') , we find α minimizing the number of non-scalar multiplications (β is computed from α); then we compute the minimum number of the required encrypted powers using the windowing technique in [43] and our optimized windowing technique, respectively.

Table II shows that compared to the windowing technique in [43], our optimized windowing technique significantly reduces the required encrypted powers. We remark that in Table II, the number of the encrypted powers using the windowing technique in [43] does not decrease with the target depth d' . As

we stated before, the reason is that the windowing technique of [43] introduces a lower bound on the number of powers.

Remark. Sending fewer powers does not always imply less running time because the server must compute these powers using non-scalar multiplications, resulting in additional computational cost. Therefore, the main influence of our optimization is that we allow more tradeoffs between communication and computation. In particular, if the protocol is run in a low-bandwidth network, our optimization may result in less running time.

C. Retrieval of Blocks

We show how to extend our idea to the setting that the database entries are bitstrings. We use the idea of [7], which views a database with σ -bit entries as σ subdatabases with entries being bits. Let $DB = (DB_1, \dots, DB_n) \in \{0, 1\}^{\sigma \times n}$ be a database where each $DB_j = (DB_{j,1}, \dots, DB_{j,\sigma}) \in \{0, 1\}^\sigma$, then we can define σ subdatabases $\{DB^k\}_{k \in [\sigma]}$ where each $DB^k = (DB_{1,k}, \dots, DB_{n,k}) \in \{0, 1\}^n$. Then to retrieve DB_i , the client runs a PIR protocol for each subdatabase DB^k . We note that the messages sent by the client can be used in every execution because it has the same input for each execution. Therefore, we can appropriately increase the workload of the client to reduce the dependence of the cost on σ .

V. IMPLEMENTATION

We fully implemented BitBatSPIR and compared it to the current state-of-the-art BatchPIR protocol VecBatchPIR [24]. It is our intention to open-source the code after publication.

For a fair comparison, we chose to implement BitBatSPIR (based on the implementation of the underlying PSI protocols [40], [42], [43]) and VecBatchPIR in a unified manner. Specifically, we first studied the open-source implementations of [40], [42], [43]⁷ and VecBatchPIR⁸, including linking libraries, implementation details, and parameter selections. Then, we re-implemented these schemes and BitBatSPIR using Java with the computational security parameter $\kappa = 128$ and the statistical security parameter $\lambda = 40$. Note that all schemes require operations related to homomorphic encryption that strongly affect the efficiencies. We used the Java Native Interface (JNI) technique to invoke the BFV scheme [47], [51] implemented in Microsoft SEAL library in C/C++, where we chose the latest version v4.1 with default CMake options.

A. Implementation Details

In our implementation, we split BitBatSPIR and VecBatchPIR into offline and online phases. In the offline phase, the server and the client execute procedures independent of the query indices, while the actual query is executed in the online phase. In the following, we list implementation details for both protocols.

BitBatSPIR. In the offline phase, the client generates keys related to homomorphic encryption and sends parts of (public)

keys to the server, including the relinearization keys for multiplication operations. Sending these keys is the only communication cost involved in that phase. By contrast, the server needs more work to preprocess entries in the database. We introduce the following optimizations in our implementation.

First, we follow [43] and use the Elliptic Curve Diffie-Hellman-based OPRF protocol of [41], [54]. In short, this OPRF requires the server to independently sample an OPRF key, hash each index to a uniformly random elliptic curve group element (we treat the hash as a random oracle), and multiply this group element with the sampled key. We stress that all procedures are independent of the client's retrieval indices so that they can be executed in the offline phase. As in [43], we use the FourQ elliptic curve [64] to speed up the hash-to-curve and the scalar multiplication operations.

Second, the server partitions indices into bins, where the number of bins depends on the maximal number of indices the client queries in a batch. The size of items mapped to any given bin can be made public since the OPRF is applied and the bin size will not leak any information to the client.

In the online phase, the server and the client execute OPRF, where the server takes the pre-sampled OPRF key, and the client takes the retrieval indices as inputs. Then, the client maps its retrieval indices to the corresponding OPRF outputs, runs PSI with the server, and computes the retrieved entries from the obtained intersection. In our implementation, we fix the query powers sent by the client for all queries. We note that we can further trade-off between computational cost and communication cost by using the windowing technique (Section IV-B), i.e., adaptively choosing the query powers per query.

VecBatchPIR. In the offline phase, the client also sends parts of (public) keys to the server. We note that VecBatchPIR additionally requires Galois keys for rotation operations. The server first uses stash-less cuckoo Hashing with 3 hash functions to insert entries into $(1 + \epsilon)n$ bins, where $\epsilon = 0.2$. Then, for entries in each bin, the server encodes them in three dimensions, and within a plaintext vector, entries are placed at some specific slots. Finally, the server transfers the plaintexts into NTT form. The communication cost is the serialized Galois keys and relinearization keys generated by the client.

In the online phase, the client updates the original retrieval indices and obtains one index for each bin. Then, the client generates queries for each bin and merges them to reduce the communication cost. The server generates a response and merges the ciphertexts by rotation operations.

B. Evaluations and Comparisons

We run experiments on a single Intel Core i9-9900K with 3.6GHz and 128GB RAM. We simulate the network connection using Linux command `tc`. All experiments are run in LAN (10Gbps bandwidth and 0.05ms RTT latency) and WAN (10Mbps bandwidth with 80ms RTT latency) network settings.

Our experiments consider two database sizes (2^{20} , 2^{24}), with batch sizes (256, 512, 1024, 2048, 4096) and (1024, 2048, 4096, 8192) respectively. We separately report

⁷<https://github.com/microsoft/APSII>

⁸https://github.com/mhmughees/vectorized_batchpir

TABLE III
THE EFFICIENCY COMPARISON BETWEEN **VecBatchPIR** AND **BitBatSPIR** IN THE LAN SETTING. BETTER RESULTS FOR **BitBatSPIR** ARE MARKED WITH BOLD TEXT.

Network (LAN)		10Gbps Bandwidth and 0.05ms RTT Latency							
		Offline Comm. (MB)		Offline Time (s)		Online Comm. (MB)		Online Time (s)	
Database Size	Batch Size	[24]	Ours	[24]	Ours	[24]	Ours	[24]	Ours
2^{20}	256	17.875	0.120	6.613	27.842	1.264	1.678	1.157	1.931
	512	17.875	0.127	5.736	25.774	1.895	2.580	1.363	2.278
	1024	17.875	0.135	11.015	24.174	1.895	2.122	1.443	2.074
	2048	17.875	0.128	9.650	24.893	3.159	3.906	2.029	3.006
	4096	17.875	0.310	9.056	24.438	6.317	4.981	3.026	3.101
2^{24}	1024	17.875	0.706	61.101	727.747	6.317	2.663	8.200	10.902
	2048	17.875	0.706	85.989	721.531	6.317	3.908	7.942	11.647
	4096	17.875	0.706	88.546	724.769	12.635	6.397	11.060	13.854
	8192	17.875	0.706	96.666	693.631	24.637	13.504	17.359	19.757

TABLE IV
THE EFFICIENCY COMPARISON BETWEEN **VecBatchPIR** AND **BitBatSPIR** IN THE WAN SETTING. BETTER RESULTS FOR **BitBatSPIR** ARE MARKED WITH BOLD TEXT.

Network (WAN)		10Mbps Bandwidth and 80ms RTT Latency							
		Offline Comm. (MB)		Offline Time (s)		Online Comm. (MB)		Online Time (s)	
Database Size	Batch Size	[24]	Ours	[24]	Ours	[24]	Ours	[24]	Ours
2^{20}	256	17.876	0.120	21.891	27.722	1.263	1.672	2.512	3.902
	512	17.875	0.135	20.972	27.233	1.896	2.128	3.083	5.089
	1024	17.875	0.135	25.162	24.650	1.895	2.122	3.247	4.464
	2048	17.875	0.128	24.853	25.910	3.159	3.908	4.854	6.881
	4096	17.875	0.310	24.676	25.589	6.318	4.981	8.959	7.836
2^{24}	1024	17.875	0.706	76.497	727.240	6.317	2.663	14.155	13.634
	2048	17.875	0.706	98.883	721.251	6.317	3.908	13.750	15.536
	4096	17.875	0.706	102.882	727.450	12.635	6.397	22.163	19.978
	8192	17.875	0.706	110.287	689.444	24.639	13.504	38.791	31.992

communication costs and running time for the offline and online phases for comparisons.

The experiment results in the LAN setting are given in Table III. When the database size is small (2^{20}), **BitBatSPIR** has a much lower offline communication cost. With the database size increases (to 2^{24}), **BitBatSPIR** further achieves lower online communication cost, about half of **VecBatchPIR**. For high-speed network settings like LAN, the communication cost has less impact on the total running time. Even though, the online running time of **BitBatSPIR** is only $1.024 \sim 1.481 \times$ slower than that of **VecBatchPIR**.

Table IV shows the comparisons in the WAN setting. In this network setting, **BitBatSPIR** has a better running time when the database size is 2^{24} . This is because the asymptotic communication cost of **BitBatSPIR** is $\text{poly}(\log n)$, compared to $O(\sqrt[3]{n})$ for **VecBatchPIR**. To conclude, **BitBatSPIR** is more friendly to large database sizes with relatively low-speed network settings.

VI. CONCLUSION AND FUTURE WORK

In this work, we presented an efficient black-box construction of **BatchSPIR** from **PSI**. By using an efficient **PSI**

protocol in the unbalanced setting, we obtained a new lattice-based **BatchSPIR** protocol that enjoys better efficiency. We implemented our protocol for databases with bit entries and compared it to the state-of-the-art **PIR** protocol in the batch setting. Our experimental results showed that our protocol achieves not only a stronger security guarantee (symmetric privacy) but also a higher efficiency, especially for large databases and in the WAN setting.

Our work introduced a new framework for practical **PIR** and **SPIR**. To date, most lattice-based **PIR** and **SPIR** protocols (including **VecBatchPIR** [24]) are based on the hierarchical **PIR** framework of Kushilevitz and Ostrovsky [9], that is, these **PIRs** represent the database as a d -dimensional hypercube. **BitBatSPIR** showed a new framework for constructing practical **PIR** and **SPIR**. Our new framework and the hierarchical **PIR** framework are complementary, in the sense that our framework is more suitable for databases with small entries and the hierarchical **PIR** framework is more suitable for databases with large entries.

Possible Optimizations via Size-Hiding PSI. In our **BatchSPIR** construction, we must prevent the client from knowing the

size of the server's set $L = \{j \in [n] : \text{DB}_j = 1\}$. To achieve this, the server pads L using elements from $[n+1, 2n]$. However, this method increases the total cost, especially when the database contains a small number of ones. A possible approach for improving this is to use size-hiding PSI [65] that hides the sizes of the sets. However, existing size-hiding PSI protocols [65], [66] are designed for the balanced setting and have linear communication in the sizes of both sets. Similar to the unbalanced PSI protocol of [43], an unbalanced size-hiding PSI would help further optimize our SPIR protocol.

Higher efficiency for PIR. Our SPIR protocol naturally implies a PIR protocol. However, our approach can be further optimized when we only want a PIR protocol. The first thing to notice is that we no longer need to protect the privacy of the database, so the server no longer needs to pad its input set. Second, in our SPIR protocol, we let the server collect the indexes with entry 1 as its input set L . Note that the size of L affects the efficiency of the protocol. We can have the server collect the sets of indexes with entries 0 and 1, respectively, and then use the smaller set as its own input set. In order for the client to know the entry corresponding to the server input set, the server simply hands the entry (0 or 1) corresponding to its own input set to the client set, which only adds one bit of communication. This method allows us to reduce the upper bound on the size of the server's input set from n to $\lfloor n/2 \rfloor$, thereby increasing the efficiency of the resulting PIR protocol.

REFERENCES

- [1] L. Sassaman, B. Cohen, and N. Mathewson, "The pynchon gate: a secure method of pseudonymous mail retrieval," in *WPES 2005*, 2005.
- [2] S. Angel and S. T. V. Setty, "Unobservable communication over fully untrusted infrastructure," in *OSDI 2016*, 2016.
- [3] S. Angel, H. Chen, K. Laine, and S. T. V. Setty, "PIR with compressed queries and amortized query processing," in *SP 2018*, 2018.
- [4] W. Lueks and I. Goldberg, "Sublinear scaling for multi-client private information retrieval," in *FC 2015*, 2015.
- [5] T. Gupta, N. Crooks, W. Mulhern, S. T. V. Setty, L. Alvisi, and M. Walfish, "Scalable and private media consumption with popcorn," in *NSDI 2016*, 2016.
- [6] A. Ali, T. Lepoint, S. Patel, M. Raykova, P. Schoppmann, K. Seth, and K. Yeo, "Communication-computation trade-offs in PIR," in *USENIX Security 2021*, 2021.
- [7] B. Chor, O. Goldreich, E. Kushilevitz, and M. Sudan, "Private information retrieval," in *FOCS 1995*, 1995.
- [8] B. Chor, E. Kushilevitz, O. Goldreich, and M. Sudan, "Private information retrieval," *J. ACM*, 1998.
- [9] E. Kushilevitz and R. Ostrovsky, "Replication is NOT needed: SINGLE database, computationally-private information retrieval," in *FOCS 1997*, 1997.
- [10] C. Cachin, S. Micali, and M. Stadler, "Computationally private information retrieval with polylogarithmic communication," in *EUROCRYPT 1999* (J. Stern, ed.), vol. 1592 of *Lecture Notes in Computer Science*, pp. 402–414, Springer, 1999.
- [11] Y. Chang, "Single database private information retrieval with logarithmic communication," in *ACISP 2004* (H. Wang, J. Pieprzyk, and V. Varadharajan, eds.), vol. 3108 of *Lecture Notes in Computer Science*, pp. 50–61, Springer, 2004.
- [12] C. Gentry and Z. Ramzan, "Single-database private information retrieval with constant communication rate," in *ICALP 2005* (L. Caires, G. F. Italiano, L. Monteiro, C. Palamidessi, and M. Yung, eds.), vol. 3580 of *Lecture Notes in Computer Science*, pp. 803–815, Springer, 2005.
- [13] R. Sion and B. Carbunar, "On the practicality of private information retrieval," in *NDSS 2007*, 2007.
- [14] C. A. Melchor, J. Barrier, L. Fousse, and M. Killijian, "XPIR : Private information retrieval for everyone," *PoPETS 2016*, 2016.
- [15] C. Gentry and S. Halevi, "Compressible FHE with applications to PIR," in *TCC 2019*, 2019.
- [16] J. Park and M. Tibouchi, "SHECS-PIR: somewhat homomorphic encryption-based compact and scalable private information retrieval," in *ESORICS 2020*, 2020.
- [17] M. H. Mughees, H. Chen, and L. Ren, "Onionpir: Response efficient single-server PIR," in *CCS 2021*, 2021.
- [18] I. Ahmad, Y. Yang, D. Agrawal, A. E. Abbadi, and T. Gupta, "Addr: Metadata-private voice communication over fully untrusted infrastructure," in *OSDI 2021*, 2021.
- [19] S. J. Menon and D. J. Wu, "SPIRAL: fast, high-rate single-server PIR via FHE composition," in *SP 2022*, 2022.
- [20] Y. Gertner, Y. Ishai, E. Kushilevitz, and T. Malkin, "Protecting data privacy in private information retrieval schemes," in *STOC 1998*, 1998.
- [21] C. Lin, Z. Liu, and T. Malkin, "XSPIR: efficient symmetrically private information retrieval from ring-lwe," in *ESORICS 2022*, 2022.
- [22] Y. Ishai, E. Kushilevitz, R. Ostrovsky, and A. Sahai, "Batch codes and their applications," in *STOC 2004* (L. Babai, ed.), pp. 262–271, ACM, 2004.
- [23] A. Beimel, Y. Ishai, and T. Malkin, "Reducing the servers computation in private information retrieval: PIR with preprocessing," in *CRYPTO 2000* (M. Bellare, ed.), vol. 1880 of *Lecture Notes in Computer Science*, pp. 55–73, Springer, 2000.
- [24] M. H. Mughees and L. Ren, "Vectorized batch private information retrieval," in *SP 2023*, pp. 437–452, IEEE, 2023.
- [25] C. A. Meadows, "A more efficient cryptographic matchmaking protocol for use in the absence of a continuously available third party," in *SP 1986*, 1986.
- [26] B. A. Huberman, M. K. Franklin, and T. Hogg, "Enhancing privacy and trust in electronic communities," in *EC 1999*, 1999.
- [27] C. Dong, L. Chen, and Z. Wen, "When private set intersection meets big data: an efficient and scalable protocol," in *CCS 2013*, 2013.
- [28] B. Pinkas, T. Schneider, and M. Zohner, "Faster private set intersection based on OT extension," in *USENIX Security 2014*, 2014.
- [29] B. Pinkas, T. Schneider, G. Segev, and M. Zohner, "Phasing: Private set intersection using permutation-based hashing," in *USENIX Security 2015*, 2015.
- [30] V. Kolesnikov, R. Kumaresan, M. Rosulek, and N. Trieu, "Efficient batched oblivious PRF with applications to private set intersection," in *CCS 2016*, 2016.
- [31] P. Rindal and M. Rosulek, "Improved private set intersection against malicious adversaries," in *EUROCRYPT 2017*, 2017.
- [32] B. Pinkas, T. Schneider, and M. Zohner, "Scalable private set intersection based on OT extension," *ACM Trans. Priv. Secur.*, 2018.
- [33] B. Pinkas, M. Rosulek, N. Trieu, and A. Yanai, "Spot-light: Lightweight private set intersection from sparse OT extension," in *CRYPTO 2019*, 2019.
- [34] B. Pinkas, M. Rosulek, N. Trieu, and A. Yanai, "PSI from paxos: Fast, malicious private set intersection," in *EUROCRYPT 2020*, 2020.
- [35] M. Chase and P. Miao, "Private set intersection in the internet setting from lightweight oblivious PRF," in *CRYPTO 2020*, 2020.
- [36] P. Rindal and P. Schoppmann, "VOLE-PSI: fast OPRF and circuit-psi from vector-ole," in *EUROCRYPT 2021*, 2021.
- [37] G. Garimella, B. Pinkas, M. Rosulek, N. Trieu, and A. Yanai, "Oblivious key-value stores and amplification for private set intersection," in *CRYPTO 2021*, 2021.
- [38] M. Rosulek and N. Trieu, "Compact and malicious private set intersection for small sets," in *CCS 2021*, 2021.
- [39] Á. Kiss, J. Liu, T. Schneider, N. Asokan, and B. Pinkas, "Private set intersection for unequal set sizes with mobile applications," *PoPETS 2017*, 2017.
- [40] H. Chen, K. Laine, and P. Rindal, "Fast private set intersection from homomorphic encryption," in *CCS 2017*, 2017.
- [41] A. C. D. Resende and D. F. Aranha, "Faster unbalanced private set intersection," in *FC 2018*, 2018.
- [42] H. Chen, Z. Huang, K. Laine, and P. Rindal, "Labeled PSI from fully homomorphic encryption with malicious security," in *CCS 2018*, 2018.
- [43] K. Cong, R. C. Moreno, M. B. da Gama, W. Dai, I. Iliashenko, K. Laine, and M. Rosenberg, "Labeled PSI from homomorphic encryption with reduced computation and communication," in *CCS '21*, 2021.
- [44] B. Chor, N. Gilboa, and M. Naor, "Private information retrieval by keywords," *IACR Cryptol. ePrint Arch.*, p. 3, 1998.
- [45] C. Gentry, "Fully homomorphic encryption using ideal lattices," in *STOC 2009*, 2009.
- [46] Z. Brakerski, C. Gentry, and V. Vaikuntanathan, "(leveled) fully homomorphic encryption without bootstrapping," in *ITCS 2012*, 2012.

- [47] J. Fan and F. Vercauteren, “Somewhat practical fully homomorphic encryption,” *IACR Cryptol. ePrint Arch.*, 2012.
- [48] J. W. Bos, K. E. Lauter, J. Loftus, and M. Naehrig, “Improved security for a ring-based fully homomorphic encryption scheme,” in *IMACC 2013*, 2013.
- [49] C. Gentry, A. Sahai, and B. Waters, “Homomorphic encryption from learning with errors: Conceptually-simpler, asymptotically-faster, attribute-based,” in *CRYPTO 2013*, 2013.
- [50] Z. Brakerski and V. Vaikuntanathan, “Efficient fully homomorphic encryption from (standard) LWE ,” *SIAM J. Comput.*, 2014.
- [51] Z. Brakerski, “Fully homomorphic encryption without modulus switching from classical gapsvp,” in *CRYPTO 2012*, 2012.
- [52] C. Gentry, S. Halevi, and N. P. Smart, “Homomorphic evaluation of the AES circuit,” in *CRYPTO 2012*, 2012.
- [53] A. Kim, Y. Polyakov, and V. Zucca, “Revisiting homomorphic encryption schemes for finite fields,” in *ASIACRYPT 2021*, 2021.
- [54] S. Jarecki and X. Liu, “Fast secure computation of set intersection,” in *SCN 2010* (J. A. Garay and R. D. Prisco, eds.), vol. 6280 of *Lecture Notes in Computer Science*, pp. 418–435, Springer, 2010.
- [55] R. Pagh and F. F. Rodler, “Cuckoo hashing,” in *ESA 2001* (F. M. auf der Heide, ed.), vol. 2161 of *Lecture Notes in Computer Science*, pp. 121–133, Springer, 2001.
- [56] L. Devroye and P. Morin, “Cuckoo hashing: Further analysis,” *Inf. Process. Lett.*, vol. 86, no. 4, pp. 215–219, 2003.
- [57] D. Fotakis, R. Pagh, P. Sanders, and P. G. Spirakis, “Space efficient hash tables with worst case constant access time,” in *STACS 2003* (H. Alt and M. Habib, eds.), vol. 2607 of *Lecture Notes in Computer Science*, pp. 271–282, Springer, 2003.
- [58] N. P. Smart and F. Vercauteren, “Fully homomorphic encryption with relatively small key and ciphertext sizes,” in *PKC 2010* (P. Q. Nguyen and D. Pointcheval, eds.), vol. 6056 of *Lecture Notes in Computer Science*, pp. 420–443, Springer, 2010.
- [59] N. P. Smart and F. Vercauteren, “Fully homomorphic simd operations,” *Des. Codes Cryptogr.*, vol. 71, no. 1, pp. 57–81, 2014.
- [60] M. Paterson and L. J. Stockmeyer, “On the number of nonscalar multiplications necessary to evaluate polynomials,” *SIAM J. Comput.*, 1973.
- [61] M. F. Challis, “Two New Techniques for Computing Extremal h -bases A_k ,” *The Computer Journal*, 1993.
- [62] M. F. Challis, H. House, and J. P. Robinson, “Some extremal postage stamp bases,” 2010.
- [63] C. Gentry, S. Halevi, and N. P. Smart, “Fully homomorphic encryption with polylog overhead,” in *EUROCRYPT 2012*, 2012.
- [64] C. Costello and P. Longa, “Fourq: four-dimensional decompositions on a q -curve over the mersenne prime.” *Cryptology ePrint Archive*, Paper 2015/565, 2015.
- [65] G. Ateniese, E. D. Cristofaro, and G. Tsudik, “(if) size matters: Size-hiding private set intersection,” in *PKC 2011* (D. Catalano, N. Fazio, R. Gennaro, and A. Nicolosi, eds.), vol. 6571 of *Lecture Notes in Computer Science*, pp. 156–173, Springer, 2011.
- [66] Y. Zhan, Z. Zhang, Q. Liu, and B. Wang, “Hiding the input-size in multi-party private set intersection,” *Des. Codes Cryptogr.*, pp. 2893–2915, 2023.